Cache Simulation System

8-Way Block Set Associative | LRU Replacement Policy | Analysis Writeup

Project Links

Resource	Link
Web App	https://simulationproject.onrender.com/
Walkthrough Video	(paste your video link here)
Source Code	https://github.com/DLSU-Garcia/CSC512C-Gamboa-Garcia-Simulation-Project

System Specifications

Parameter	Value
Cache Mapping Type	8-Way Block Set Associative
Replacement Policy	LRU (Least Recently Used)
Number of Ways	8
Number of Sets	2
Total Cache Blocks (n)	16 blocks (8 ways × 2 set)
Cache Line Size	2 words per block
Main Memory Blocks	1024 blocks (fixed)
Read Policy	Non Load-Through
Cache Hit Time	1 ns
Main Memory Access Time	10 ns
Miss Penalty	1 ns (cache access) + 10ns x block size x 1 ns = 1ns+10ns*2*1ns=22ns

Notes on Configuration

Address Decomposition

With 1024 memory blocks, 2 sets, and 2 words per line, the address fields are:

```
Memory blocks  = 1024 = 2^10  →  Block address field = 10 bits
Sets            =    2 = 2^1   →  SET field            =  1 bits
Words per line = 2 = 2^1   →  WORD (offset) field =  1 bits
TAG field       = 10 - 1 - 1        →  TAG field        = 8 bits
Full address   = TAG[8] | SET[1] | WORD[1] = 10 bits total
```

Address Breakdown Example — Memory Block 7

Block 7 in binary: `0000000111` (10 bits)

Field	Bits	Bit Count	Note
TAG	00000001	8	
SET	1	1	Only 2 sets
WORD	1	1	2 words in one block

Block 7 can map to **any of the 8 cache slots of Set 1 (7%2=1)**. Placement is governed by LRU ordering.

How 8-Way BSA + LRU Works

An 8-way Block Set Associative Cache works as a hybrid between a Direct Mapped Cache and a Full Associative Cache. For this instance, we have 2 cache sets that contain 8 slots each that are available to memory blocks that are mapped to that particular cache set. They are mapped based on their Block number. Cache Set Assignment = Block number modulo 2(number of sets). For example, MM Block 7 maps to Set 1 because 7 modulo 2 = 1.

On a Cache Hit

- **The block is found** in one of the 8 slots in either cache set 0 or 1.
- Its LRU position is updated — it becomes the **Most Recently Used (MRU)**.
- Data is served from cache in 1 ns.

On a Cache Miss (Non Load-Through)

- All 2 cache sets and their 8 slots are checked — the block is not present.
- If a free slot exists, the block is loaded there.

- If all 8 slots are full, the **Least Recently Used block is evicted**.
- The new block takes the evicted slot and becomes MRU.
- Data is served from cache (not directly from memory).

LRU Ordering

The cache tracks recency as a queue: leftmost = LRU (evict next), rightmost = MRU (most recent).

```
[LRU ← oldest]   Slot0 | Slot1 | Slot2 | ... | Slot7   [→ MRU: most recent]
```

Timing Model

Event	Time
Cache Hit	1 ns
Miss Penalty	1 ns (cache access) + 10ns x block size + 1 ns = 1ns +10ns x 2 +1ns = 22ns
Average Access Time	(Hit Rate × 1) + (Miss Penalty × Miss Rate) ns
Total Access Time	(# of blocks hit * block size) * read cache time + [read cache time + transfer lines to cache + transfer cache to cpu] x # of misses

Test Case A — Sequential Sequence

Sequence: `0,1,2,...,31, 0,1,2,...,31`

Up to 2n = 32 blocks, repeated 2 times. Total accesses: 64.

Access Trace

1. Block 0: MISS - loaded into set 0, block 0

2. Block 1: MISS - loaded into set 1, block 0

3. Block 2: MISS - loaded into set 0, block 1

4. Block 3: MISS - loaded into set 1, block 1

5. Block 4: MISS - loaded into set 0, block 2

6. Block 5: MISS - loaded into set 1, block 2

7. Block 6: MISS - loaded into set 0, block 3

8. Block 7: MISS - loaded into set 1, block 3

9. Block 8: MISS - loaded into set 0, block 4

10. Block 9: MISS - loaded into set 1, block 4

11. Block 10: MISS - loaded into set 0, block 5

12. Block 11: MISS - loaded into set 1, block 5

13. Block 12: MISS - loaded into set 0, block 6

14. Block 13: MISS - loaded into set 1, block 6

15. Block 14: MISS - loaded into set 0, block 7

16. Block 15: MISS - loaded into set 1, block 7

17. Block 16: MISS - evicted block 0 from set 0, block 0

18. Block 17: MISS - evicted block 1 from set 1, block 0

19. Block 18: MISS - evicted block 2 from set 0, block 1

20. Block 19: MISS - evicted block 3 from set 1, block 1

21. Block 20: MISS - evicted block 4 from set 0, block 2

22. Block 21: MISS - evicted block 5 from set 1, block 2

23. Block 22: MISS - evicted block 6 from set 0, block 3

24. Block 23: MISS - evicted block 7 from set 1, block 3

25. Block 24: MISS - evicted block 8 from set 0, block 4

26. Block 25: MISS - evicted block 9 from set 1, block 4

27. Block 26: MISS - evicted block 10 from set 0, block 5

28. Block 27: MISS - evicted block 11 from set 1, block 5

29. Block 28: MISS - evicted block 12 from set 0, block 6

30. Block 29: MISS - evicted block 13 from set 1, block 6

31. Block 30: MISS - evicted block 14 from set 0, block 7

32. Block 31: MISS - evicted block 15 from set 1, block 7

33. Block 0: MISS - evicted block 16 from set 0, block 0

34. Block 1: MISS - evicted block 17 from set 1, block 0

35. Block 2: MISS - evicted block 18 from set 0, block 1

36. Block 3: MISS - evicted block 19 from set 1, block 1

37. Block 4: MISS - evicted block 20 from set 0, block 2

38. Block 5: MISS - evicted block 21 from set 1, block 2

39. Block 6: MISS - evicted block 22 from set 0, block 3

40. Block 7: MISS - evicted block 23 from set 1, block 3

41. Block 8: MISS - evicted block 24 from set 0, block 4

42. Block 9: MISS - evicted block 25 from set 1, block 4

43. Block 10: MISS - evicted block 26 from set 0, block 5

44. Block 11: MISS - evicted block 27 from set 1, block 5

45. Block 12: MISS - evicted block 28 from set 0, block 6

46. Block 13: MISS - evicted block 29 from set 1, block 6

47. Block 14: MISS - evicted block 30 from set 0, block 7

48. Block 15: MISS - evicted block 31 from set 1, block 7

49. Block 16: MISS - evicted block 0 from set 0, block 0

50. Block 17: MISS - evicted block 1 from set 1, block 0

51. Block 18: MISS - evicted block 2 from set 0, block 1

52. Block 19: MISS - evicted block 3 from set 1, block 1

53. Block 20: MISS - evicted block 4 from set 0, block 2

54. Block 21: MISS - evicted block 5 from set 1, block 2

55. Block 22: MISS - evicted block 6 from set 0, block 3

56. Block 23: MISS - evicted block 7 from set 1, block 3

57. Block 24: MISS - evicted block 8 from set 0, block 4

58. Block 25: MISS - evicted block 9 from set 1, block 4

59. Block 26: MISS - evicted block 10 from set 0, block 5

60. Block 27: MISS - evicted block 11 from set 1, block 5

61. Block 28: MISS - evicted block 12 from set 0, block 6

62. Block 29: MISS - evicted block 13 from set 1, block 6

63. Block 30: MISS - evicted block 14 from set 0, block 7

64. Block 31: MISS - evicted block 15 from set 1, block 7

Statistics

Metric	Value
Memory Access Count	64
Cache Hit Count	0
Cache Miss Count	64
Cache Hit Rate	0.00%
Cache Miss Rate	100.00%
Average Memory Access Time	(0 × 1) + (22.00 × 1) ns = 22.00 ns
Total Memory Access Time	(0 * 2) * 1ns + [1ns+ 1*2 + 10ns*2] x 64 1472 ns

Analysis

Test Case A produces a **100% miss rate** even with 8-way BSA. The sequence accesses 2n = 32 distinct blocks, which exceeds the 16 cache blocks' capacity. By the time the second pass

begins (access #33 onward), all blocks from the first half have been evicted by the second half. LRU faithfully evicts the oldest block on each miss, but since every new block is unique and the working set (32 blocks) exceeds the cache size (16 blocks), no hits are ever possible. This is the **thrashing** scenario for LRU — the working set is exactly twice the cache capacity.

Test Case B — Mid-Repeat Blocks

Sequence: `0,1,...,15, 1,2,...,15, 16,...,31, 0,1,...,15, 1,2,...,15, 16,...,31`

Start at 0; repeat blocks 1–(n-1) twice; continue to 2n; repeat the entire sequence twice. Total accesses: 94.

Access Trace

1. Block 0: MISS - loaded into set 0, block 0
2. Block 1: MISS - loaded into set 1, block 0
3. Block 2: MISS - loaded into set 0, block 1
4. Block 3: MISS - loaded into set 1, block 1
5. Block 4: MISS - loaded into set 0, block 2
6. Block 5: MISS - loaded into set 1, block 2
7. Block 6: MISS - loaded into set 0, block 3
8. Block 7: MISS - loaded into set 1, block 3
9. Block 8: MISS - loaded into set 0, block 4
10. Block 9: MISS - loaded into set 1, block 4
11. Block 10: MISS - loaded into set 0, block 5
12. Block 11: MISS - loaded into set 1, block 5
13. Block 12: MISS - loaded into set 0, block 6
14. Block 13: MISS - loaded into set 1, block 6
15. Block 14: MISS - loaded into set 0, block 7
16. Block 15: MISS - loaded into set 1, block 7
17. **Block 1: HIT (in cache set 1, block 0)**

18. **Block 2: HIT (in cache set 0, block 1)**

19. **Block 3: HIT (in cache set 1, block 1)**

20. **Block 4: HIT (in cache set 0, block 2)**

21. **Block 5: HIT (in cache set 1, block 2)**

22. **Block 6: HIT (in cache set 0, block 3)**

23. **Block 7: HIT (in cache set 1, block 3)**

24. **Block 8: HIT (in cache set 0, block 4)**

25. **Block 9: HIT (in cache set 1, block 4)**

26. **Block 10: HIT (in cache set 0, block 5)**

27. **Block 11: HIT (in cache set 1, block 5)**

28. **Block 12: HIT (in cache set 0, block 6)**

29. **Block 13: HIT (in cache set 1, block 6)**

30. **Block 14: HIT (in cache set 0, block 7)**

31. **Block 15: HIT (in cache set 1, block 7)**

32. Block 16: MISS - evicted block 0 from set 0, block 0

33. Block 17: MISS - evicted block 1 from set 1, block 0

34. Block 18: MISS - evicted block 2 from set 0, block 1

35. Block 19: MISS - evicted block 3 from set 1, block 1

36. Block 20: MISS - evicted block 4 from set 0, block 2

37. Block 21: MISS - evicted block 5 from set 1, block 2

38. Block 22: MISS - evicted block 6 from set 0, block 3

39. Block 23: MISS - evicted block 7 from set 1, block 3

40. Block 24: MISS - evicted block 8 from set 0, block 4

41. Block 25: MISS - evicted block 9 from set 1, block 4

42. Block 26: MISS - evicted block 10 from set 0, block 5

43. Block 27: MISS - evicted block 11 from set 1, block 5

44. Block 28: MISS - evicted block 12 from set 0, block 6

45. Block 29: MISS - evicted block 13 from set 1, block 6

46. Block 30: MISS - evicted block 14 from set 0, block 7

47. Block 31: MISS - evicted block 15 from set 1, block 7

48. Block 0: MISS - evicted block 16 from set 0, block 0

49. Block 1: MISS - evicted block 17 from set 1, block 0

50. Block 2: MISS - evicted block 18 from set 0, block 1

51. Block 3: MISS - evicted block 19 from set 1, block 1

52. Block 4: MISS - evicted block 20 from set 0, block 2

53. Block 5: MISS - evicted block 21 from set 1, block 2

54. Block 6: MISS - evicted block 22 from set 0, block 3

55. Block 7: MISS - evicted block 23 from set 1, block 3

56. Block 8: MISS - evicted block 24 from set 0, block 4

57. Block 9: MISS - evicted block 25 from set 1, block 4

58. Block 10: MISS - evicted block 26 from set 0, block 5

59. Block 11: MISS - evicted block 27 from set 1, block 5

60. Block 12: MISS - evicted block 28 from set 0, block 6

61. Block 13: MISS - evicted block 29 from set 1, block 6

62. Block 14: MISS - evicted block 30 from set 0, block 7

63. Block 15: MISS - evicted block 31 from set 1, block 7

64. Block 1: HIT (in cache set 1, block 0)

65. Block 2: HIT (in cache set 0, block 1)

66. Block 3: HIT (in cache set 1, block 1)

67. Block 4: HIT (in cache set 0, block 2)

68. Block 5: HIT (in cache set 1, block 2)

69. Block 6: HIT (in cache set 0, block 3)

70. **Block 7: HIT (in cache set 1, block 3)**

71. **Block 8: HIT (in cache set 0, block 4)**

72. **Block 9: HIT (in cache set 1, block 4)**

73. **Block 10: HIT (in cache set 0, block 5)**

74. **Block 11: HIT (in cache set 1, block 5)**

75. **Block 12: HIT (in cache set 0, block 6)**

76. **Block 13: HIT (in cache set 1, block 6)**

77. **Block 14: HIT (in cache set 0, block 7)**

78. **Block 15: HIT (in cache set 1, block 7)**

79. Block 16: MISS - evicted block 0 from set 0, block 0

80. Block 17: MISS - evicted block 1 from set 1, block 0

81. Block 18: MISS - evicted block 2 from set 0, block 1

82. Block 19: MISS - evicted block 3 from set 1, block 1

83. Block 20: MISS - evicted block 4 from set 0, block 2

84. Block 21: MISS - evicted block 5 from set 1, block 2

85. Block 22: MISS - evicted block 6 from set 0, block 3

86. Block 23: MISS - evicted block 7 from set 1, block 3

87. Block 24: MISS - evicted block 8 from set 0, block 4

88. Block 25: MISS - evicted block 9 from set 1, block 4

89. Block 26: MISS - evicted block 10 from set 0, block 5

90. Block 27: MISS - evicted block 11 from set 1, block 5

91. Block 28: MISS - evicted block 12 from set 0, block 6

92. Block 29: MISS - evicted block 13 from set 1, block 6

93. Block 30: MISS - evicted block 14 from set 0, block 7

94. Block 31: MISS - evicted block 15 from set 1, block 7

Statistics

Metric	Value
Memory Access Count	94
Cache Hit Count	30
Cache Miss Count	64
Cache Hit Rate	31.91%
Cache Miss Rate	68.09%
Average Memory Access Time	(0.30 × 1) + (22.00 × 0.64) ns 15.30 ns
Total Memory Access Time	(30 * 2) * 1ns + [1ns+ 1*2 + 10ns*2] x 64 1532 ns

Analysis

Test Case B is the **best-performing** scenario. The mid-repeat subsequence (blocks 1–15 accessed immediately after the initial load) falls entirely **within the 2 cache sets' 16-slot capacity**. At access #16, all 8 blocks (0–15) are resident in cache, so the 15 repeated accesses generate **15 hits per pass** (30 total across 2 passes). LRU correctly keeps the recently-accessed blocks alive while pushing block 0 toward the eviction head. Once the sequence moves to blocks 16–30 the 16-slot limit is exceeded and misses resume. The 31.91% hit rate is entirely attributable to the mid-repeat locality window.

Test Case C — Random Sequence

Sequence (64 unique blocks):

```
 1. Block 39: MISS - loaded into set 1, block 0
 2. Block 220: MISS - loaded into set 0, block 0
 3. Block 364: MISS - loaded into set 0, block 1
 4. Block 730: MISS - loaded into set 0, block 2
 5. Block 722: MISS - loaded into set 0, block 3
 6. Block 229: MISS - loaded into set 1, block 1
 7. Block 571: MISS - loaded into set 1, block 2
 8. Block 209: MISS - loaded into set 1, block 3
 9. Block 463: MISS - loaded into set 1, block 4
10. Block 869: MISS - loaded into set 1, block 5
11. Block 90: MISS - loaded into set 0, block 4
12. Block 472: MISS - loaded into set 0, block 5
13. Block 399: MISS - loaded into set 1, block 6
14. Block 3: MISS - loaded into set 1, block 7
```

15. Block 244: MISS - loaded into set 0, block 6
16. Block 973: MISS - evicted block 39 from set 1, block 0
17. Block 75: MISS - evicted block 229 from set 1, block 1
18. Block 629: MISS - evicted block 571 from set 1, block 2
19. Block 334: MISS - loaded into set 0, block 7
20. Block 367: MISS - evicted block 209 from set 1, block 3
21. Block 991: MISS - evicted block 463 from set 1, block 4
22. Block 577: MISS - evicted block 869 from set 1, block 5
23. Block 434: MISS - evicted block 220 from set 0, block 0
24. Block 764: MISS - evicted block 364 from set 0, block 1
25. Block 456: MISS - evicted block 730 from set 0, block 2
26. Block 105: MISS - evicted block 399 from set 1, block 6
27. Block 534: MISS - evicted block 722 from set 0, block 3
28. Block 941: MISS - evicted block 3 from set 1, block 7
29. Block 34: MISS - evicted block 90 from set 0, block 4
30. Block 180: MISS - evicted block 472 from set 0, block 5
31. Block 191: MISS - evicted block 973 from set 1, block 0
32. Block 19: MISS - evicted block 75 from set 1, block 1
33. Block 325: MISS - evicted block 629 from set 1, block 2
34. Block 310: MISS - evicted block 244 from set 0, block 6
35. Block 198: MISS - evicted block 334 from set 0, block 7
36. Block 368: MISS - evicted block 434 from set 0, block 0
37. Block 12: MISS - evicted block 764 from set 0, block 1
38. Block 249: MISS - evicted block 367 from set 1, block 3
39. Block 746: MISS - evicted block 456 from set 0, block 2
40. Block 436: MISS - evicted block 534 from set 0, block 3
41. Block 325: HIT (in cache set 1, block 2)
42. Block 892: MISS - evicted block 34 from set 0, block 4
43. Block 758: MISS - evicted block 180 from set 0, block 5
44. Block 530: MISS - evicted block 310 from set 0, block 6
45. Block 181: MISS - evicted block 991 from set 1, block 4
46. Block 142: MISS - evicted block 198 from set 0, block 7
47. Block 133: MISS - evicted block 577 from set 1, block 5
48. Block 750: MISS - evicted block 368 from set 0, block 0
49. Block 452: MISS - evicted block 12 from set 0, block 1
50. Block 16: MISS - evicted block 746 from set 0, block 2
51. Block 981: MISS - evicted block 105 from set 1, block 6
52. Block 143: MISS - evicted block 941 from set 1, block 7
53. Block 300: MISS - evicted block 436 from set 0, block 3
54. Block 951: MISS - evicted block 191 from set 1, block 0
55. Block 574: MISS - evicted block 892 from set 0, block 4
56. Block 74: MISS - evicted block 758 from set 0, block 5
57. Block 809: MISS - evicted block 19 from set 1, block 1
58. Block 647: MISS - evicted block 249 from set 1, block 3

```
59. Block 476: MISS - evicted block 530 from set 0, block 6
60. Block 176: MISS - evicted block 142 from set 0, block 7
61. Block 47: MISS - evicted block 325 from set 1, block 2
62. Block 601: MISS - evicted block 181 from set 1, block 4
63. Block 445: MISS - evicted block 133 from set 1, block 5
64. Block 694: MISS - evicted block 750 from set 0, block 0
```

Statistics

Metric	Value
Memory Access Count	64
Cache Hit Count	1
Cache Miss Count	63
Cache Hit Rate	1.6%
Cache Miss Rate	98.4%
Average Memory Access Time	(0.016 × 1) + (22.00 × 0.984) ns = 21.67 ns
Total Memory Access Time	(1 * 2) * 1ns + [1ns+ 1*2 + 10ns*2] x 63 = 1451 ns

Analysis

A random sequence of 64 blocks that is selected from the entire 1024 Main Memory Block selection pool unsurprisingly resulted in 63 out of 64 blocks being **uniquely accessed, with only 1 block being accessed twice**. Since LRU (or any replacement policy) can only produce hits on repeat accesses, and there is only 1, the result is a 98.4% miss rate. The 8-way BSA, as with any cache mapping scheme, provides no advantage when there is zero locality. The first 16 accesses fill the cache; every subsequent access evicts the LRU block and loads a new one, which is never seen again. This is a **primarily cold-start** workload due to the low probability of cache hits.

Comparative Analysis

Metric	Test A (Sequential)	Test B (Mid-Repeat)	Test C (Random)
Total Accesses	64	94	64
Hits	0	30	1
Misses	64	64	63
Hit Rate	0.00%	31.91%	1.6%

Miss Rate	100.00%	68.09%	98.4%
Avg Access Time	22.00 ns	15.30 ns	21.67 ns
Total Access Time	1472 ns	1532 ns	1451 ns

Key Observations

- **LRU thrashes when the working set exceeds cache capacity.** In Test A, 32 blocks compete for 16 slots. LRU evicts the oldest block on each miss, but the sequential pattern guarantees that evicted block will be needed again soon, thus the classic LRU thrashing scenario.
- **LRU shines when the working set fits within cache.** In Test B, the mid-repeat windows fit perfectly in 16 slots. LRU promotes hits to MRU and keeps them alive, generating 15 consecutive hits per pass.
- **8-way BSA nearly eliminates conflict misses entirely.** Unlike direct mapping, every block can occupy any slot in its assigned cache set. Misses are primarily compulsory (cold) or capacity misses — rarely conflict misses.
- **Random unique sequences are difficult for any replacement policy.** Without repeated accesses, no policy can generate hits. Cache performance is limited purely by the access pattern, not the architecture.
- **Comparison with Direct Mapping (4 blocks).** The 8-way BSA achieves a slightly better mid-repeat hit rate (31.91% vs 27.3%) and mostly eliminates the conflict misses that affected direct mapping. However, both suffer 0-1.6% hit rates on sequential and random workloads respectively where working set or mostly unique-block count exceeds capacity.

Application Features

- **Configurable Parameters —** Cache blocks (min 4, power of 2), words per line (min 2, power of 2), and number of ways are all user-adjustable.
- **LRU State Visualization —** The cache display shows all 8 ways in LRU-to-MRU order, with color highlighting for hits, misses, and evictions.
- **Step-by-Step Animated Tracing —** Step through each memory access one at a time, watching the LRU queue update in real time.
- **Final Snapshot Mode —** Skip directly to the final cache state after all accesses.
- **Text Log —** Full access-by-access log showing block number, tag, hit/miss result, evicted block (if any), and full cache state.
- **Statistics Panel —** Displays all required output metrics: memory access count, hit count, miss count, hit rate, miss rate, average access time, and total access time.
- **Three Built-in Test Cases —** Sequential, Mid-Repeat, and Random sequences are pre-loaded and selectable from the interface.

Full Specifications Summary

Parameter	Specification
Cache Mapping	8-Way Block Set Associative (BSA)
Replacement Policy	LRU (Least Recently Used)
Number of Ways	8
Number of Sets	2
Cache Line Size	2 words (configurable, min 2, power of 2)
Total Cache Blocks	16 blocks (configurable, min 4, power of 2)
Memory Blocks	1024 blocks (fixed)
Read Policy	Non Load-Through
Cache Hit Time	1 ns
Memory Access Time	10 ns
Address Format	Full address = TAG[8] | SET[1] | WORD[1] = 10 bits total
Programming Stack	Python, NiceGUI
Platform	Render